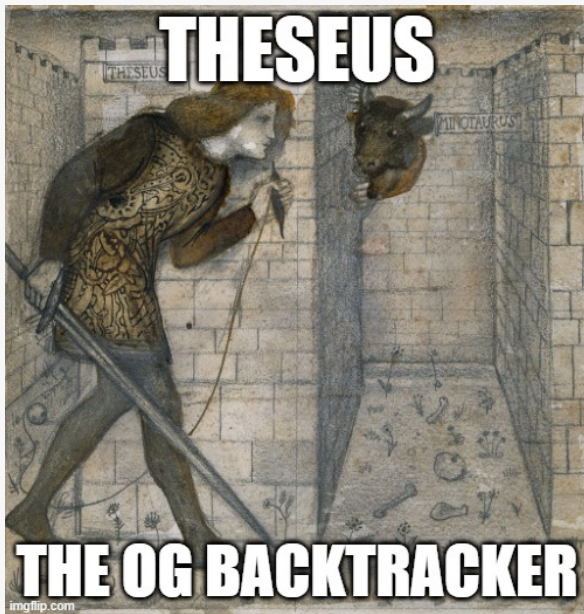




General Training for Programming Competitions & Coding Interviews

November 20, 2024

Backtracking, Divide-and-Conquer, Dynamic Programming, and Greedy Algorithms

Markus Fischer, Paul G. Rump, Christoph Staudt

Faculty of Mathematics and Computer Science



What's this Lecture About?

In this lecture, we cover some more **strategies for algorithm design**:

- Handle repetitive tasks through **iteration** (→ loops),
- Iterate elegantly using **recursion** (→ recursive structure),
- Use **brute force** to be lazy but powerful (→ exhaustive search),

What's this Lecture About?

In this lecture, we cover some more **strategies for algorithm design**:

- Handle repetitive tasks through **iteration** (→ loops),
- Iterate elegantly using **recursion** (→ recursive structure),
- Use **brute force** to be lazy but powerful (→ exhaustive search),
- Test bad options, then **backrack**,
- **Divide and conquer** your toughest opponents,
- **Dynamically** piece together the solution,
- Save time with **heuristics** for a reasonable way out.

Backtracking

Backtracking = ‘Rücksetzverfahren’

Calculate some or all solutions by **incrementally building candidates**. Abandon a candidate (‘backtrack’) if it can no longer be completed to a valid solution.

Basically an exhaustive **depth-first search**, with **pruning**. Backtracking problems often have **exponential / factorial runtimes**.

Backtracking = 'Rücksetzverfahren' 🤪

Calculate some or all solutions by **incrementally building candidates**. Abandon a candidate ('backtrack') if it can no longer be completed to a valid solution.

Basically an exhaustive **depth-first search**, with **pruning**. Backtracking problems often have **exponential / factorial runtimes**.

An iteration of the **general backtracking algorithm** works as follows. At each step, we start from a given partial solution, say, $a = (a_1, a_2, \dots, a_k)$, and extend it by adding another element a_{k+1} at the end. Then we test whether what we have so far is a solution.

- ✓ If so, we're done (unless we want multiple solutions).
- ✗ If not, we check whether the partial solution could still become a complete solution.
 - ✓ If so, continue with $(a_1, \dots, a_{k+1})$.
 - ✗ If not, try a different possibility for a_{k+1} , if one exists.
 - ✗ If none exists, **backtrack** and try a different a_k .

Basic Slow Template For Backtracking (Overview)

backtrack_template.cpp 

```
1 // with "finished" you can kill recursion;
2 // with the "out" pointer you can get results out from "process_solution"
3 void backtrack(vector<int> &a, bool &finished, void *out) {
4     if (is_solution(a)) { // if we found a solution, process it
5         process_solution(a, out);
6     } else {
7         vector<int> candidates = construct_candidates(a);
8         for (const int candidate : candidates) {
9             a.push_back(candidate);           // add candidate to possible solution
10            backtrack(a, finished, out);       // recurse
11            a.pop_back();                       // remove candidate from solution
12            if (finished) { return; }          // terminate early
13        }
14    }
15 }
```


Basic Slow Template For Backtracking (Missing pieces)

backtrack_template.cpp 

```
1 bool is_solution(const vector<int> &a) {  
2     // This Boolean function tests whether the elements in the vector are a  
3     // complete solution for the given problem.  
4     return false;  
5 }  
6  
7 void process_solution(const vector<int> &a, void *out) {  
8     // do something with the solution  
9     return;  
10 }  
11  
12 vector<int> construct_candidates(const vector<int> &a) {  
13     vector<int> candidates;  
14     // Here, compute possible candidates for the next position of a.  
15     return candidates;  
16 }
```

Constructing All Subsets with Backtracking

We can construct the 2^n subsets of an n -item set by iterating through all possible 2^n length- n boolean vectors.

subsets.cpp 

```
1 bool is_solution(const vector<int> &a, size_t n) {
2     return a.size() == n; // test if solution size reached
3 }
4
5 void process_solution(const vector<int> &a, void *out) {
6     for (size_t i = 0; i < a.size(); ++i) {
7         if (a[i]) // if item i is in subset, print it
8             cout << i << " ";
9     }
10    cout << endl;
11 }
12
13 // It's a rather inefficient solution!
14 vector<int> construct_candidates(const vector<int> &a, size_t n) {
15     return {0, 1}; // 0 means not in subset, 1 means in subset
16 }
```

Of course, we could represent a subset as a single integer for moderate problem sizes.

Constructing Permutations with Backtracking

Print all $n!$ permutations of n items. For $n = 2$ the valid permutations are $\langle 0, 1 \rangle$ and $\langle 1, 0 \rangle$.

permutations.cpp 

```
1 bool is_solution(const vector<int> &a, int n) {  
2     return int(a.size()) == n; // test if solution size reached  
3 }  
4  
5 void process_solution(const vector<int> &a, void *out) {  
6     for (auto number : a)  
7         cout << number << " "; // print permutation  
8     cout << endl;  
9 }
```

Constructing Permutations with Backtracking (Cont.)

Print all $n!$ permutations of n items. For $n = 2$ the valid permutations are $\langle 0, 1 \rangle$ and $\langle 1, 0 \rangle$.

permutations.cpp 

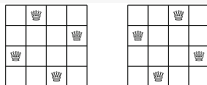
```
1 // It's a rather inefficient solution!
2 vector<int> construct_candidates(const vector<int> &a, int n) {
3     vector<int> candidates;
4     vector<bool> in_permutation(n, false); // vector of size n, all entries false
5     for (auto number : a)
6         in_permutation[number] = true;      // flag already used numbers
7     for (int i = 0; i < n; ++i)
8         if (in_permutation[i] == false)    // we prune away invalid candidates
9             candidates.push_back(i);
10    return candidates;
11 }
```

The n -Queens Problem

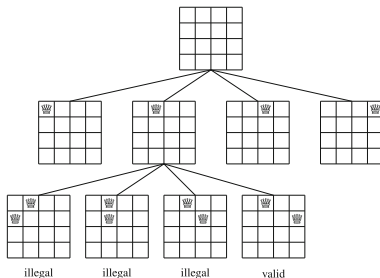
Compute the **number of ways n queens can be placed** on an $n \times n$ **chessboard** so that **no two queens attack each other**.

The n -Queens Problem

Compute the **number of ways** n queens can be placed on an $n \times n$ chessboard so that **no two queens attack each other**.



The possible ways to place 4 queens on a 4×4 chessboard



Partial solutions to the queen problem using backtracking

The n -Queens Problem Code

n_queens.cpp 

```
1 bool is_solution(const vector<int> &a, int n) {  
2     // vector a is always valid, so is_solution check is trivial  
3     return int(a.size()) == n; // test if solution size reached  
4 }  
5  
6 void process_solution(const vector<int> &a, size_t *out) {  
7     (*out)++; // increment counter, since we found a solution  
8 }
```

The n -Queens Problem Code (Continued)

n_queens.cpp 

```
1 vector<int> construct_candidates(const vector<int> &a, int n) {
2     // i-th element of the vector "a" lists the column
3     // where the i-th (row) queen resides --> possible column candidates [0, n - 1]
4     vector<int> candidates;
5     // pruning idea: no two queens may lie on the same row, column or diagonal
6     // since in vector "a" each position represents a different row, we only
7     // need to ensure that there are no conflicts on the columns and diagonals.
8     for (int i = 0; i < n; i++) {
9         bool legal_move = true;
10        for (int j = 0; j < int(a.size()); j++) {
11            if (int(a.size()) - j == abs(i - a[j])) // diagonal threat
12                legal_move = false;
13            else if (i == a[j]) // column threat
14                legal_move = false; // another queen occupies the column already
15        }
16        if (legal_move)
17            candidates.push_back(i);
18    }
19    return candidates;
20 }
```


Divide and Conquer

Divide and Conquer



A **Divide and Conquer** strategy solves a problem by **dividing** it into two (or more) **independent identical sub-problems**, ideally each about half the size of the original problem, and **solving the sub-problems recursively**.

The strategy consists in **three steps** at each level of recursion:

- a **divide phase** in which the problem is subdivided into a number of sub-problems
- a **conquer phase** where the sub-problems are solved directly (if possible) or recursively (otherwise)
- a **combine step** in which the solutions of the sub-problems are merged to obtain the solution of a bigger problem

Examples: **Quicksort**, **Mergesort**

Divide and Conquer

For most **Divide and Conquer** approaches, one of the **two** following **patterns** apply:

Divide and Conquer

For most **Divide and Conquer** approaches, one of the **two** following **patterns** apply:

- the divide phase has negligible computation cost, **most of the running time** is spent in the **combine phase** (for example merge of two sorted sub-arrays in mergesort), *or*
 - the **combine phase is totally absent** and the entire work is essentially performed in the divide phase (partitioning in quicksort).
- Either way, we want to use parallelization to increase speed!
- The **Holy Grail** of Divide and Conquer in terms of **parallelization** are approaches where **divide and combine phases are cheap** (like $\mathcal{O}(\log n)$) or absent.

Multithreaded 2-Way Merge with Divide and Conquer

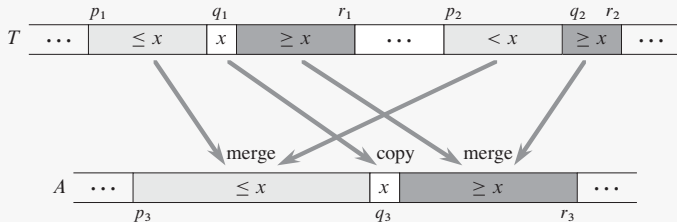
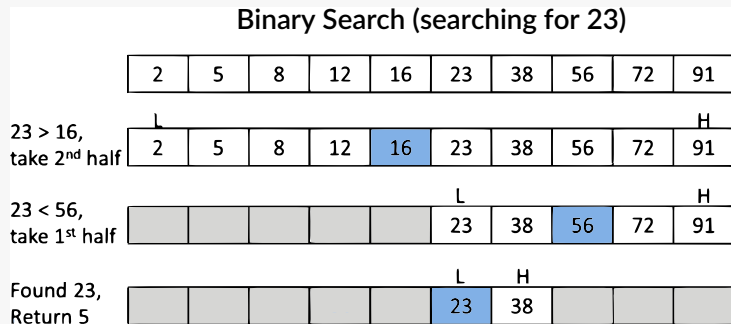
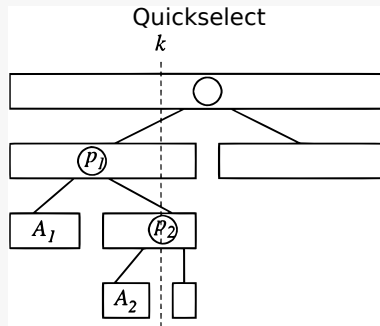


Figure 27.6 The idea behind the multithreaded merging of two sorted subarrays $T[p_1 \dots r_1]$ and $T[p_2 \dots r_2]$ into the subarray $A[p_3 \dots r_3]$. Letting $x = T[q_1]$ be the median of $T[p_1 \dots r_1]$ and q_2 be the place in $T[p_2 \dots r_2]$ such that x would fall between $T[q_2 - 1]$ and $T[q_2]$, every element in subarrays $T[p_1 \dots q_1 - 1]$ and $T[p_2 \dots q_2 - 1]$ (lightly shaded) is less than or equal to x , and every element in the subarrays $T[q_1 + 1 \dots r_1]$ and $T[q_2 + 1 \dots r_2]$ (heavily shaded) is at least x . To merge, we compute the index q_3 where x belongs in $A[p_3 \dots r_3]$, copy x into $A[q_3]$, and then recursively merge $T[p_1 \dots q_1 - 1]$ with $T[p_2 \dots q_2 - 1]$ into $A[p_3 \dots q_3 - 1]$ and $T[q_1 + 1 \dots r_1]$ with $T[q_2 \dots r_2]$ into $A[q_3 + 1 \dots r_3]$.

Introduction to Algorithms, 3rd edition, p. 798.

Decrease and Conquer

There is a class of algorithms called Divide and Conquer, but **actually they are not**. They do not split the problem into two (or more) similar sub-problems, but successively break the original problem down into **one smaller sub-problem**. (**Only one** conquer call in each recursive level.)



Dynamic Programming

DYNAMIC

"IT'S IMPOSSIBLE TO USE THE WORD 'DYNAMIC' IN THE PEJORATIVE SENSE... THUS, I THOUGHT 'DYNAMIC PROGRAMMING' WAS A GOOD NAME."

— RICHARD BELLMAN, EXPLAINING HOW HE PICKED A NAME FOR HIS MATH RESEARCH TO TRY TO PROTECT IT FROM CRITICISM (*EYE OF THE HURRICANE*, 1984)

ENTROPY

"YOU SHOULD CALL IT 'ENTROPY'... NO ONE KNOWS WHAT ENTROPY REALLY IS, SO IN A DEBATE YOU WILL ALWAYS HAVE THE ADVANTAGE."

— JOHN VON NEUMANN, TO CLAUDE SHANNON, ON WHY HE SHOULD BORROW THE PHYSICS TERM IN INFORMATION THEORY (AS TOLD TO MYRON TRIBUS)



DYNAMIC ENTROPY

SCIENCE TIP: IF YOU HAVE A COOL CONCEPT YOU NEED A NAME FOR, TRY "DYNAMIC ENTROPY."

Dynamic Programming

Like *Divide and Conquer*, **Dynamic Programming** solves the problem by combining the **solutions of multiple smaller problems**, but what makes Dynamic Programming different is that the **same sub-problem may reoccur**. Therefore, a key to making Dynamic Programming efficient is **caching the results of intermediate computations**.

Dynamic Programming

Like *Divide and Conquer*, **Dynamic Programming** solves the problem by combining the **solutions of multiple smaller problems**, but what makes Dynamic Programming different is that the **same sub-problem may reoccur**. Therefore, a key to making Dynamic Programming efficient is **caching the results of intermediate computations**.

Dynamic Programming

Like *Divide and Conquer*, **Dynamic Programming solves the problem by combining the solutions of multiple smaller problems**, but what makes Dynamic Programming different is that the **same sub-problem may reoccur**. Therefore, a key to making Dynamic Programming efficient is **caching the results of intermediate computations**.

We need to find a way to **break the original problem into sub-problems** such that we can solve the original problem relatively easily once cached solutions to the sub-problems are available. Dynamic programs are mostly implemented by **recursion**.

Alternatively, you can study the pattern of the recursive calls and implement them **iteratively**. You still 'cache' previous work. **Minimizing cache space** is a recurring theme in Dynamic Programming.

Dynamic Programming

Dynamic programs can be solved in **bottom-up** (iterative, easier to make the cache small) or **top-down** fashion (recursive, sometimes easier to implement, possibility of pruning).

Example: compute the n -th Fibonacci number.

Some people call **top-down** Dynamic Programming **memoization** and only use Dynamic Programming to refer to bottom-up work.

Fibonacci Numbers With Dynamic Programming

fibonacci.cpp 

```
1 // recursive implementation,  $O(1.618^n)$  time, 1.618 --> golden ratio
2 uint64_t fibonacci(uint64_t n) {
3     if (n <= 1) return n;
4     return fibonacci(n - 1) + fibonacci(n - 2);
5 }
6
7 // top-down dynamic programming (or memoization), with recursion,  $O(n)$  time
8 uint64_t recurse(uint64_t n, vector<uint64_t> &memo) {
9     if (n <= 1) return n;
10    if (memo[n] == 0) memo[n] = recurse(n - 1, memo) + recurse(n - 2, memo);
11    return memo[n]; // lookup number
12 }
13 uint64_t fibonacci_top_down(uint64_t n) { // call this function
14     vector<uint64_t> memo(n + 1); // create cache for intermediate results
15     return recurse(n, memo);
16 }
```

Fibonacci Numbers With Dynamic Programming

fibonacci.cpp 

```
1 // bottom-up dynamic programming, no recursion, O(n) time
2 uint64_t fibonacci_bottom_up(uint64_t n) {
3     if (n <= 1) return n;
4     vector<uint64_t> memo(n); memo[1] = 1; // create cache for intermediate results
5     for (uint64_t i = 2; i < n; i++) memo[i] = memo[i - 1] + memo[i - 2]; // compute missing number
6     return memo[n - 1] + memo[n - 2];
7 }
8
9 // bottom-up dynamic programming, O(n) time, small cache
10 uint64_t fibonacci_bottom_up_small_cache(uint64_t n) {
11     if (n == 0) return n;
12     uint64_t a = 0, b = 1; // reduce space complexity of the cache (we only need the last two numbers)
13     for (uint64_t i = 2; i < n; i++) { uint64_t c = a + b; a = b; b = c; }
14     return a + b;
15 }
```

Heuristics → in Our Case It's Being Greedy

A **heuristic** is a method that leads to a solution **without guaranteeing** that it's the best or optimal one. Heuristics can help when methods like brute force or backtracking are too slow. Heuristics can also be used within other methods (like backtracking) to speed up the computation. There are many funky heuristic approaches, but we'll focus on the simplest: **greedy**.

Heuristics → in Our Case It's Being Greedy

A **heuristic** is a method that leads to a solution **without guaranteeing** that it's the best or optimal one. Heuristics can help when methods like brute force or backtracking are too slow. Heuristics can also be used within other methods (like backtracking) to speed up the computation. There are many funky heuristic approaches, but we'll focus on the simplest: **greedy**.

The greedy approach consists in **never coming back to previous choices**. It's the **opposite of backtracking**. Try to make the **best choice at each step**, and don't question it later.
→ In other words: At each step the algorithm makes a **decision** that is **locally optimal**, and it **never changes that decision**.

A Greedy Knapsack Heuristic

A greedy burglar breaks into a museum to steal some valuable objects. He has a knapsack that can carry a weight of up to 13 (we ignore units). Which items will he steal? Remember, the less time he spends in the museum, the less likely he gets caught.



A Greedy Knapsack Heuristic

A greedy burglar breaks into a museum to steal some valuable objects. He has a knapsack that can carry a weight of up to 13 (we ignore units). Which items will he steal? Remember, the less time he spends in the museum, the less likely he gets caught.



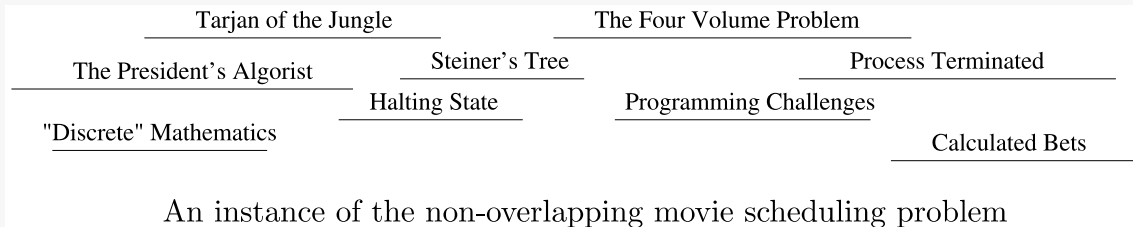
Unfortunately, there is no optimal heuristic for the 'general' 0-1 Knapsack Problem. If the weights have arbitrary precision, it's **NP-hard** ($\mathcal{O}(2^n)$). But if the weights are **positive integers** you can solve the 0-1 Knapsack Problem with dynamic programming in $\mathcal{O}(nw)$ time, where n is the number of objects and w the weight limit of the knapsack.

Non-Overlapping Movie Scheduling Problem

Problem: Movie Scheduling Problem

Input: A set I of n intervals on the line.

Output: What is the largest subset of mutually non-overlapping intervals which can be selected from I ?



An instance of the non-overlapping movie scheduling problem

Non-Overlapping Movie Scheduling Problem: Heuristics

Let's try some greedy heuristics:

EarliestJobFirst(I)

Accept the earliest starting job j from I which does not overlap any previously accepted job, and repeat until no more such jobs remain.

Non-Overlapping Movie Scheduling Problem: Heuristics

Let's try some greedy heuristics:

EarliestJobFirst(I)

Accept the earliest starting job j from I which does not overlap any previously accepted job, and repeat until no more such jobs remain.

ShortestJobFirst(I)

While ($I \neq \emptyset$) do

Accept the shortest possible job j from I .

Delete j , and any interval which intersects j from I .

The Algorithm Design Manual, 2nd edition, pp. 9–11.

How good are they?

Tarjan of the Jungle

The Four Volume Problem

The President's Algorist

Steiner's Tree

Process Terminated

"Discrete" Mathematics

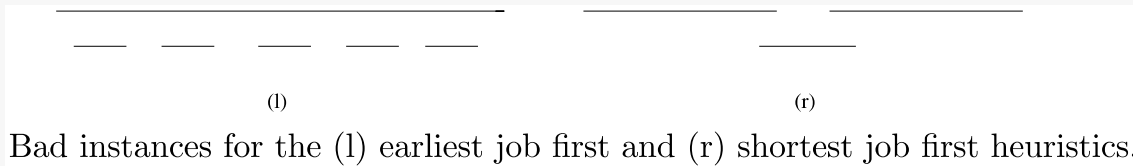
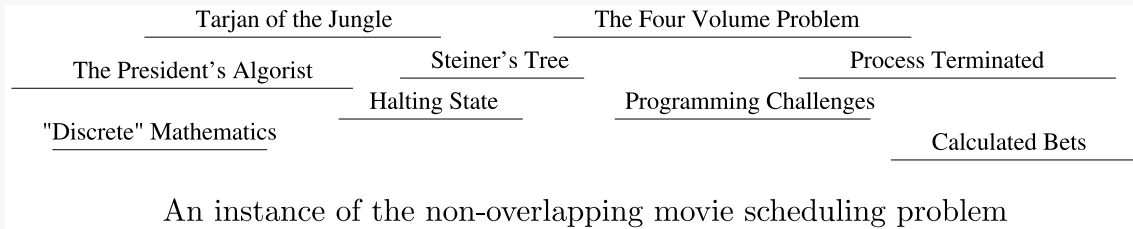
Halting State

Programming Challenges

Calculated Bets

An instance of the non-overlapping movie scheduling problem

How good are they?



Is it better not to be greedy?

At this point, an algorithm where we try all possibilities may start to look good:

ExhaustiveScheduling(I)

$j = 0$

$S_{max} = \emptyset$

 For each of the 2^n subsets S_i of intervals I

 If (S_i is mutually non-overlapping) and ($size(S_i) > j$)

 then $j = size(S_i)$ and $S_{max} = S_i$.

 Return S_{max}

Is it better not to be greedy?

At this point, an algorithm where we try all possibilities may start to look good:

ExhaustiveScheduling(I)

$j = 0$

$S_{max} = \emptyset$

 For each of the 2^n subsets S_i of intervals I

 If (S_i is mutually non-overlapping) and ($size(S_i) > j$)

 then $j = size(S_i)$ and $S_{max} = S_i$.

 Return S_{max}

But wait a minute, what about this heuristic?

OptimalScheduling(I)

 While ($I \neq \emptyset$) do

 Accept the job j from I with the earliest completion date.

 Delete j , and any interval which intersects j from I .



Exercises (Upload Solutions to your Repository)

1. Solve the **15-puzzle** with **backtracking**. 

`fifteen_puzzle.cpp` → 

2. **Read** about **dynamic programming** and **implement** one example **iteratively** and with **recursion**.

`dynamic_programming.pdf` →  `dynamic.cpp` → 

3. **Compute** the **duration** in an **optimal task assignment**.

`optimal_duration.cpp` → 

4. **Optional!** Help Karel the robot (see pdf).

`Karel_the_robot.pdf` → 

Exam Questions

- What is an optimal approach (as fast as possible) to compute the n -Queens problem with **backtracking**?
- Describe some ideas how to solve the 15-puzzle with **backtracking**.
- **Why** is **efficient parallelization not trivial** in most divide and conquer algorithms?
- That is the difference between **divide and conquer** and **decrease and conquer**?
- **What** is dynamic programming? **When** is it useful?
- **Compare** the **top-down approach** with the **bottom-up approach** used in dynamic programming.
- Briefly present a **greedy heuristic** of **your choice**.